Your codes.
Your next authenticator.

A practical guide to capturing your own iPhone's encrypted Authy tokens, recovering account identity, and moving reviewed codes into Bitwarden without waiting for an Authy support export.

Independent community tool. Observed workflow: September 10, 2026. Compatibility can change with Authy and iOS versions.

The connection has two separate jobs

1

Your iPhone

Already signed into Authy.
Standard accounts unlocked.
Backup password known.

→

2

Your computer

Wi-Fi: scoped mitmproxy capture.
USB: local backup metadata.
Offline: review + decryption.

→

3

Your Bitwarden vault

Codes attached to reviewed login IDs.
Unmatched codes kept separately.
Results read back and checked.

> A USB cable does not proxy the phone's network traffic. Both devices must be on the same reachable Wi-Fi network for the capture step.

What you need

On your devices

- Your own unlocked iPhone with Authy working.
- Your Authy encrypted-backup password.
- A computer, USB cable, trusted Wi-Fi and disk space.
- Access to your Bitwarden account.

On your computer

- Go 1.26.4+ to build the CLI.
- mitmproxy for the temporary proxy.
- libimobiledevice if using command-line backup.
- Bitwarden's `bw` CLI for automated apply.

Use your own locally generated certificate. Never install a stranger's CA, upload tokens to a converter website, or give this tool your Apple ID password.

---

01 Build and create a private workspace

From the downloaded project directory, build the tool. Homebrew users can install the optional device tools as shown.

```
go build -trimpath -o bin/authy-exit ./cmd/authy-exit
brew install mitmproxy libimobiledevice bitwarden-cli
./bin/authy-exit doctor

umask 077
WORK="$HOME/.local/share/authy-exit/session-1"
mkdir -p "$WORK"
chmod 700 "$WORK"
```

> Use a folder outside cloud-synced Downloads, Desktop and Documents. Captures contain encrypted secrets; the final export and rollback receipt contain plaintext secrets.

---

02 Connect and trust your iPhone

1. Connect the iPhone to the computer with a data-capable USB cable.
2. Unlock the phone. Approve the computer's accessory prompt if shown.
3. Open Finder and select the iPhone in the sidebar.
4. Click **Trust** on the computer, then tap **Trust** on the phone and enter its passcode there.
5. Keep the phone connected and available for further backup prompts.

Apple's Finder backup instructions cover these trust prompts. The iPhone passcode is entered on the phone; it is not the Authy backup password.

```
idevice_id -l
idevicepair -u YOUR_DEVICE_UDID pair
idevicebackup2 -u YOUR_DEVICE_UDID -i backup --full "$WORK/backup"
```

Replace the UDID with your own device from the first command. Do not use restore, encryption-off or reset commands. Backup completion may take time; wait for its success result.

---

03 Find both Wi-Fi addresses

Computer address

On macOS: System Settings → Wi-Fi → Details for the active network → TCP/IP. Read the local IP address.

If Wi-Fi is on `en0`, `ipconfig getifaddr en0` also works. Do not assume every Mac uses that interface.

iPhone address

Settings → Wi-Fi → tap the ⓘ next to the connected network → IP Address.

Use its Wi-Fi IP, not its cellular address or your router's public IP.

The examples use computer `192.168.50.10` and iPhone `192.168.50.20`. Replace them with yours. Guest-network client isolation, a VPN or a firewall can prevent the devices from reaching one another.

---

04 Start the scoped capture

```
./bin/authy-exit capture \
  --listen 192.168.50.10 --phone 192.168.50.20 --port 8088 \
  --dir "$WORK" --addon ./capture/mitm_authy.py
```

Leave this terminal running. The addon accepts the configured phone and saves only allowed encrypted records from Authy's authenticator-token route. It does not save raw traffic archives, headers, cookies or API authentication tokens. Other HTTPS hosts pass through without TLS interception.

In a second terminal, set `WORK` to the same private directory before running later commands. Avoid other browsing or account activity during the temporary proxy session.

---

05 Set the iPhone's HTTP proxy

Settings → Wi-Fi → connected network ⓘ

Scroll to **Configure Proxy** → select **Manual**

Server	Your computer's local IP, for example 192.168.50.10
Port	8088
Authentication	Off

Tap **Save**.

In iPhone Safari, visit `http://mitm.it`. This is the local proxy's certificate installation page. If it does not load, fix the IP, running listener, shared network or firewall before continuing. Do not forward the port on your router.

---

06 Install the certificate, then enable full trust

1. On `http://mitm.it`, choose the iOS certificate and allow the profile download.
2. Open Settings → **Profile Downloaded**, or General → VPN & Device Management.
3. Select the mitmproxy certificate profile, tap **Install**, and approve the phone's prompts.
4. Now go to Settings → General → About → **Certificate Trust Settings**.
5. Under **Enable Full Trust for Root Certificates**, enable trust for this mitmproxy certificate and confirm.

> **Installation and full trust are separate.** Installing the profile alone is not enough for TLS interception. Apple explains this distinction; see also mitmproxy's certificate documentation.

---

07 Let Authy synchronize

Open Authy with backups enabled and accounts unlocked. Let its normal sync run. Depending on the version, reopening the app or saving a harmless label change may produce a token sync. Restore any label change afterward. The tool captures the phone's own requests; it does not alter request versions or register a new device.

Look for `captured-encrypted-tokens.json` in the private workspace. No file means no eligible encrypted token records were captured yet. A working proxy by itself is not proof of a complete capture.

Do not reset Authy, delete accounts, disable backups, or repeatedly submit login credentials to force a capture. If this app version rejects the certificate, the tool does not bypass pinning. Use a legitimate alternate export or re-enroll the affected service.

---

08 Recover identity and review the plan

Use a completed readable local backup, or an Authy group preferences plist extracted locally from your own backup. Encrypted Finder backups must first be decrypted or extracted with a trusted tool and their separate backup password. The CLI does not decrypt them.

```
./bin/authy-exit inventory \
  --backup "$WORK/backup/YOUR_DEVICE_UDID" \
  --out "$WORK/inventory.json"

./bin/authy-exit plan \
  --capture "$WORK/captured-encrypted-tokens.json" \
  --inventory "$WORK/inventory.json" \
  --out "$WORK/plan.json"
```

Without a readable backup, omit `--inventory` and review capture-only metadata. Original logo information may be missing.

The trap we found

`account_name = Generic Blue`

This is an icon/theme category. Treating it as a username produces misleading verification-code entries.

The evidence to use

Explicit username → original label → original issuer → original logo → owner's confirmation.

A logo helps identify a provider; it does not prove which login owns a seed.

Open `plan.json` locally. Confirm each included entry's name, issuer, username and website URLs, then set `reviewed: true`. Leave unknown usernames empty. Set `exclude: true` for unwanted or unavailable tokens. Duplicate names stay separate by token ID.

An entry without a captured seed is unresolved, not a successful export. Some Authy-managed accounts do not return portable seeds and need service re-enrollment.

---

09 Decrypt and check codes

```
./bin/authy-exit export \
  --capture "$WORK/captured-encrypted-tokens.json" \
  --plan "$WORK/plan.json" --dry-run

./bin/authy-exit export \
  --capture "$WORK/captured-encrypted-tokens.json" \
  --plan "$WORK/plan.json" --out "$WORK/bitwarden.json"

./bin/authy-exit verify \
  --capture "$WORK/captured-encrypted-tokens.json" \
  --plan "$WORK/plan.json" --id YOUR_TOKEN_ID
```

Enter the Authy backup password into the hidden terminal prompt. It is not the iPhone passcode, Finder backup password or Bitwarden master password. Do not put passwords in command arguments.

All included records must decrypt and validate before any export is written. The verify command privately compares a phone code with the generated code. Repeat after the code changes. Later, test actual service sign-ins before retiring Authy entries.

---

10 Attach to the correct Bitwarden logins

Use the official CLI to sign in and unlock your intended vault. The session variable remains in your shell; do not paste it into chat or a report.

```
bw login
export BW_SESSION="$(bw unlock --raw)"
bw status
```

Create a private `targets.json` object mapping an Authy source token ID to its confirmed existing Bitwarden item ID. Use `{}` to keep every source as a standalone entry.

```
{"synthetic-1": "00000000-0000-4000-8000-000000000099"}
```

Replace the following user and server with the exact values from your own `bw status`. First run the preview:

```
./bin/authy-exit apply \
  --file "$WORK/bitwarden.json" --targets "$WORK/targets.json" \
  --expected-user you@example.invalid \
  --expected-server https://vault.bitwarden.com
```

After reviewing the result, run the same command with:

```
  --receipt "$WORK/apply-receipt-1.json" --yes
```

The tool preserves existing login data, adds the TOTP and source marker, and places changed entries in **Verification codes**. Unmatched sources become separate items. Different existing codes and conflicting mappings block the run. Source markers make completed writes recognizable on retry.

> Apply is not an all-or-nothing server transaction. Keep the private receipt if interrupted, preview again, and use a new receipt filename for remaining work. Avoid simultaneous vault edits. The receipt includes original passwords and must stay private.

Alternatively, use Bitwarden's JSON import UI. That creates new items and does not attach codes to existing logins. Choose one import route. Bitwarden's import format and CLI documentation explain the destination tools.

---

11 Verify, then dismantle the temporary connection

1. Compare codes and test real service sign-ins for each account you will retire from Authy.
2. On iPhone: Settings → Wi-Fi → network ⓘ → Configure Proxy → **Off** → Save.
3. Confirm normal phone connectivity, then stop the capture terminal with Ctrl-C.
4. Disable full trust for the temporary mitmproxy certificate and remove its profile under General → VPN & Device Management.
5. Lock Bitwarden and clear the shell session variable.
6. Retain or remove private capture/export/receipt files according to your backup needs. Deletion does not erase copies in snapshots or cloud backups.

```
bw lock
unset BW_SESSION
```

If additional capture is still needed, keep the listener running while the phone's proxy remains configured. Do not stop the computer-side listener first.

Troubleshooting

Symptom	Check next
Phone not detected over USB	Unlock, use a data cable, approve Trust, confirm the intended UDID.
mitm.it does not load	Computer IP and port, running listener, same Wi-Fi, guest isolation, firewall and VPN.
Certificate installed but TLS fails	Enable full trust separately. If the app rejects the certificate afterward, stop; compatibility is not guaranteed.
Capture has fewer records than the inventory	Allow normal sync and rebuild the plan. Missing managed seeds may require re-enrollment.
Decryption fails	Check the Authy backup password and capture completeness. An old password or damaged/nonstandard record can fail validation.

Symptom	Check next
Generic Blue / unknown site	Use original metadata and owner review; never invent a username or infer a website from a generic icon.
Bitwarden already has another TOTP	Confirm which enrollment works. The tool intentionally refuses to overwrite a different code.
Phone loses Wi-Fi after capture	Turn Configure Proxy off, or restart the listener if more capture is intended.

About the method

The observed legacy envelope uses PBKDF2-HMAC-SHA1 and AES-256-CBC. The program performs local decryption with the owner's known backup password. It does not recover an unknown password or defeat account authentication. Standard TOTP test vectors come from RFC 6238.

Authy Exit is MIT licensed. The implementation and examples contain no owner's capture, account list, credentials, or proprietary app code. Climaker's CLI Printing Press was used for hand-built CLI checks; this is an independently implemented local workflow, not an official Authy export feature.

Source references: mitmproxy modes · domain filtering · device backup CLI · Authy decryption prior art.